

Resumo HTML5 & CSS3 - Módulo 2

Boas práticas

Você pode colocar min-width, min-height, max-width e max-height no CSS dos seus elementos. Inclusive, se você não quer que uma imagem se achate demais porque se não ela ficará feia, você pode colocar o max-width nela.

Usando seu seletor geral para as CSS (*), você pode configurar o padding e margin com 0px e ir colocando os paddings e margins de cada elemento no seu devido CSS. Além disso, vá sempre fazendo cada coisinha e a testando.

Protótipo

Sempre faça um protótipo do seu site, não importa se vai demorar e é trabalhoso. O protótipo do site é uma parte **fundamental** e não deve ser pulada. Em primeiro lugar, pense no layout, imagens, cores e tipografia que você usará. Além disso, pense na experiência do usuário. Você pode criar seus protótipos no Figma, ou no MockFlow (<https://mockflow.com/>).

@ = indica uma regra.

Cores

As cores têm um papel fundamental no nosso site. Elas fazem com que o usuário queira estar no nosso site e navegar mais um pouco ou não, sendo que o usuário, muitas vezes, toma essa decisão instintivamente. Com frequência, nós vemos coisas com cores que no conjunto, achamos bonitas, mas não sabemos explicar o porquê. Isso ocorre devido a **psicologia das cores**.

Sempre que você for fazer um site, parta de uma cor principal e logo depois as cores que a acompanharão no seu projeto. Cada cor traz consigo um sentimento e significado, você deve pesquisar sobre elas e ver o que o seu projeto quer transmitir para o usuário. Além disso, existem várias estratégias de montar sua paleta de cores,

como as cores análogas (cores parecidas), entre outras que nós temos no pdf sobre cores. Você pode montar sua paleta de cores no Adobe Color. Use de 3 a 5 cores, sem contar preto e branco.

Para inserir cores, nós temos a sua representação por, as mais comuns:

- `rgb(00, 00, 00)` - aonde os números vão de 0 até 255;
- `rgba(00, 00, 00, 0.000)` - alfa (transparência);
- hexadecimal (`#00ABA9`) - aonde vai de 0 até F, sendo 0 o menos significativo e F o mais significativo;
- entre outras.

Um site com um fundo muito preto e as letras brancas pode dar muito cansaço visual.

No caso de fazer um site no modo noturno, tente colocar um preto mais leve.

Degradê

O degradê em um elemento deve ser colocado com `“background-image: linear-gradient(90deg, #color, #color)”`. Você pode colocar porcentagem nas cores logo atrás delas (`#color 90%`) e pode mudar a forma que você direciona o gradient indicando por uma palavra (`90deg = to right`).

Além do `“linear-gradient”`, também há o `“radial-gradient”` onde você coloca o formato do seu degradê. Exemplo: `background-image: radial-gradient(circle, red 10%, yellow 40%, green 100%)`.

Coloque sempre um fundo com alguma cor semelhante ao seu degradê pelo atributo `“background-color”` caso seu degradê falhe.

Fontes

Os navegadores têm, por padrão, a fonte Times New Roman aplicada no seu site se você não configurar fonte nenhuma. Tipos de fontes:

- Serifada: melhor no papel;
- Sem-serifa: ideal para sites;
- Monoespçada: todas as letras têm a mesma largura lateral;
- Manuscritas: parecem escritas à mão;

- Display (comemorativa).

Use de 3 a 4 fontes no seu site, no máximo. Você pode tentar descobrir qual fonte é pelo whatfontis e encontrar ela no Google Fonts, Dafont ou similares. No Google Fonts, você precisa importar ela no CSS e já usar o nome dela em um elemento. No Dafont, você precisa salvar o arquivo da fonte e configurar a fonte primeiro. As seguintes configurações devem ser feitas:

```
@font-face {  
  font-family: 'Love'; /*Você pode colocar o nome que você quiser.*/  
  src: url('fonts/love larry ttf.ttf');  
  font-weight: normal; /*Isso aqui mostra que a fonte funciona melhor no normal.*/  
  font-style: normal;  
}
```

Tome cuidado com a particularidade de cada fonte. As fontes do tipo .ttf e .otf pegam melhor.

Por padrão, as letras no site têm 16px. 16px = 1em = 1rem. Escolha usar 1em ou 1rem ao invés de pixels, pois elas são medidas relativas e não absolutas. Com medidas absolutas, no nosso dispositivo pode ficar bom, mas, em outros, pode não ficar tão legal.

Diferenças entre “rem” e “em”:

- **1rem** é sempre fixo e relativo ao elemento raiz (<html>). Se o <html> for 16px, 1rem = 16px.
- **1em** é relativo ao elemento **pai** onde ele está. Se a fonte do elemento pai for 20px, 1em = 20px. Só serão iguais se o pai tiver a mesma fonte do <html>.

Nós também podemos também podemos deixar 10px = 1rem da seguinte forma:

```
CSS  
  
html {  
  font-size: 62.5%; /* Isso transforma 1rem em exatamente 10px! */  
}  
  
body {  
  font-size: 1.6rem; /* Equivale aos 16px padrão */  
}  
  
h1 {  
  font-size: 3.2rem; /* Agora fica fácil: 3.2rem = 32px */  
}
```

O **font-weight** pode ser colocado da forma escrita (light, normal, bold...) ou em números (de 100 a 900).

Shorthand para font: style > weight > size > family. Exemplo: *font: italic bolder 1.3em 'Work Sans', sans-serif.*

Quando você for colocar uma fonte que o nome dela tem duas palavras, você deve colocá-la entre aspas simples. Exemplo: *'Work Sans'.*

Span

A tag span não tem um significado semântico em si. Ela funciona como se fosse uma div porém sendo do tipo *"inline"*. Ela não tem um resultado visual, mas pode ser utilizada para separar uma palavra e estilizá-la com CSS ou mudar seu comportamento com JavaScript. Exemplos que ela é usada:

- No CSS, em etiquetas de categoria em blogs ou contadores de notificações;
- No JS, se você precisa que um número ou palavra mude dinamicamente na tela, você isola com o *span* e coloca um *id* para o JS a localizar.

Pseu-class

Usado em tags. Exemplo: *a:hover, a:active; a:visited.*

Pseu-element

É acompanhado de dois pontos antes (::). Utilizado para:

- Colocar ícones ou setas na frente de itens de uma lista ou de botões;
- Adicionar aspas automáticas antes de um bloco de citação;
- Colocar um asterisco vermelho (*) na frente de campos obrigatórios em um formulário.

Você pode deixar ainda mais específico em qual elemento do seu HTML você quer mexer. Exemplo:

```

CSS

/* Coloca uma setinha automática apenas nos links que têm rel="external" */
a[rel="external"]::after {
  content: " ↗";
}

```

Além disso, você pode tirar aquela setinha do lado da tag <summary>.

```

CSS

/* Esconde a setinha padrão do navegador */
summary::marker {
  content: "";
}

```

Site de símbolos: <https://erikasarti.com/html/dingbats-simbolos-desenhos/> para você usar com a tecnologia pseu-element.

O que você colocar dentro da propriedade content não poderá ser selecionado para o usuário para, por exemplo, copiar. Além disso, os robôs de busca do Google e ferramentas para ajudar pessoas com deficiência não detectam conteúdo escrito dentro do “content”. Por isso, nunca coloque informações cruciais para o entendimento da página dentro do content. Use-o apenas para decorações, ícones informativos visuais, aspas ou setas.

Shorthands

- **Border:** width > style > color;
- **Outline:** width > style > color;
- **Padding:** 10px (cima e baixo) auto (laterais) // 10px (top) 20px (right) 30px (bottom) 5px (left) // 10px (todos);
- **Margin:** 10px (cima e baixo) auto (laterais) // 10px (top) 20px (right) 30px (bottom) 5px (left) // 10px (todos); (pode ser colocado margem negativa)
- **Border-radius:** 10px (top left – bottom right) 50px (top right – bottom left) // 10px (todos).

OBS: você pode colocar o border radius um por um também. Exemplo: *border-bottom-right-radius: 5px.*

Outline

A principal diferença entre a border e o outline é que o outline não ocupa espaço no layout.

- Se você colocar uma border: 20px; em um elemento, ele vai empurrar todos os elementos ao redor para abrir espaço para ela.
- Se você colocar um outline: 20px;, ele será desenhado por cima de tudo o que estiver em volta (inclusive invadindo o espaço da margin ou de outros elementos vizinhos), sem mover um único pixel do layout do seu site.

Shadow

Para a criação de sombras, tanto para letras quanto para caixas, você não precisa deixar uma sombra extremamente evidente, o nosso cérebro já percebe uma pouca diferença ali. Para isso, use a propriedade alfa (transparência) do rgb. Ela já funciona como uma shorthand, sendo que suas medidas são: *lado > baixo > espalhamento > cor*. Só existe sombra preta.

P: A tag <p> tem margem automática de 16px.

Grouping Tags

As caixas como div e spans não são obsoletas. Porém, as grouping tags são semânticas e facilitam a acessibilidade. Grouping tags: header, main, footer, article, aside, nav e section. Você pode colocar qualquer uma dessas caixas dentro de qualquer uma dessas caixas.

Aside

Aside é um conteúdo periférico do artigo, um conteúdo relativo a ele. Não se trata do conteúdo que principal que está sendo falado na sua página. Exemplos:

- **Sidebars (Barras laterais):** Menus secundários, links para os "posts mais lidos", caixas de busca ou redes sociais do site.

- **Boxes de destaque (Callouts):** No meio de um artigo de notícias, sabe aquele bloco com a biografia do autor, um glossário explicando um termo difícil ou uma curiosidade extra? Isso é um `<aside>`.
- **Publicidade:** Como os anúncios não fazem parte do texto principal (se você remover o anúncio, o artigo ainda faz sentido sozinho), eles se encaixam perfeitamente na definição da tag.

Leitores de tela para pessoas com deficiência visual usam tags como `<aside>` para permitir que o usuário "pule" direto para o conteúdo principal ou ignore blocos secundários. Usar o `<aside>` para anúncios e sidebars ajuda muito a organizar a navegação para essas ferramentas!

Article = artigo.

O `<article>` representa um bloco de conteúdo que é **completamente autônomo**. Se você recortasse esse bloco da página e colasse em outro site, ele ainda faria sentido sozinho.

- **Pense como:** Um artigo de jornal, uma matéria de revista, um post de blog, ou um comentário de um usuário.
- **Regra de ouro:** Se você puder distribuir esse conteúdo via RSS ou postar ele sozinho em outra página e as pessoas conseguirem entender o contexto completo, ele é um `<article>`.

Faz sentido sozinho no mundo? ➡ `<article>`

Section = usado para seção.

É uma parte/capítulo de algo maior e tem um título próprio? ➡ `<section>`

Quando usar section e quando usar div?

Essa é uma das dúvidas mais clássicas do HTML! A diferença entre elas não é visual (as duas se comportam exatamente igual no CSS), mas sim semântica (o significado que elas dão ao código).

A regra de ouro para decidir é: A informação ali dentro tem relevância para o conteúdo ou serve apenas para organizar o visual?

Quando usar <section>?

Você deve usar a <section> quando estiver agrupando um conteúdo que tem um tema em comum e que faz parte de uma divisão lógica da sua página.

Pense como capítulos de um livro: Se a sua página fosse um livro, as <section> seriam os capítulos ou subseções estruturadas.

A regra do Título: Quase toda <section> deve ter um título interno (<h2>, <h3>, etc.).

Se o bloco de conteúdo não faz sentido ter um título, provavelmente ele não deveria ser uma section.

Exemplos clássicos:

- Seção "Sobre Nós" de um site;
- Seção de "Depoimentos de Clientes";
- Seção de "Preços/Planos";
- Abas de um painel de controle (Seção "Perfil", Seção "Segurança").

Quando usar <div>?

A <div> (de division) não tem nenhum significado semântico. Ela é apenas uma caixa vazia. Você deve usá-la puramente para fins de estilização com CSS, organização de layout ou ganchos de JavaScript.

Pense como uma sacola plástica: Ela serve apenas para embrulhar as coisas para você conseguir movê-las ou pintá-las juntas.

Exemplos clássicos:

- Agrupar elementos para usar Flexbox ou CSS Grid (ex: uma div para alinhar dois botões lado a lado);
- Criar uma caixa de fundo colorido ou com bordas personalizadas ao redor de um texto;
- Criar um wrapper ou container para centralizar o conteúdo do site na tela.

Variáveis

Para criar variáveis, você deve colocá-las no campo root, colocar dois traços antes do nome delas (ex: --), nomeá-las e colocar o seu valor (cor, fonte, etc).

Position

A propriedade position do CSS serve para ditar como um elemento deve ser posicionado na tela. É ela quem define se um elemento vai seguir o fluxo natural do texto, se vai ficar flutuando, ou se vai ficar travado em um lugar específico quando você rola a página.

Por padrão, você não consegue mover as coisas livremente com propriedades de direção (top, bottom, left, right) a menos que você mude o position do elemento.

Existem 5 valores principais que você precisa dominar:

1. position: static (O padrão)

É o comportamento nativo de qualquer elemento no HTML. Eles seguem o fluxo normal da página: blocos ficam um embaixo do outro e elementos de linha ficam lado a lado.

Detalhe crucial: Se você tentar usar top, bottom, left ou right em um elemento static, o navegador vai ignorar completamente. Eles não têm efeito aqui.

2. position: relative (O ponto de partida ou o ajuste fino)

Quando você aplica relative, o elemento continua ocupando o seu espaço original no fluxo da página, mas agora você ganha superpoderes para duas coisas:

Mover a partir de onde ele já estava: Se você colocar top: 20px;, ele vai descer 20px em relação à sua própria posição original (sem empurrar os vizinhos, ele se move como um "fantasma").

Servir de âncora: Ele vira a referência de espaço para qualquer elemento filho que use position: absolute (exatamente o que está acontecendo na imagem que você enviou!).

Sendo caracterizado como relative, o elemento também se mantém adaptável caso tenha mudanças no tamanho do navegador.

3. position: absolute (O elemento livre)

O elemento é "arrancado" do fluxo normal da página. É como se ele passasse a flutuar em uma camada acima de tudo. Os outros elementos agem como se ele nem existisse e ocupam o espaço que era dele.

Como ele se move: Ele usa as propriedades top, bottom, left e right para se posicionar em relação ao primeiro elemento pai que tiver um position diferente de static (geralmente um pai com position: relative).

No arquivo word sobre vídeo: O iframe está com absolute e top: 5%. Ele vai ficar grudado 5% abaixo do topo da div.video (que é o pai relativo dele), e não do topo da tela inteira.

4. position: fixed (O travado na tela)

Muito parecido com o absolute, ele também flutua acima do layout, mas com uma diferença crucial: a referência dele é sempre a janela do navegador (viewport).

Onde é usado: Sabe aquele botão de "Voltar ao topo" ou aquele chat de suporte que fica preso no canto inferior direito, e não importa o quanto você role a página, ele continua fixo ali? Isso é position: fixed.

Ele também tem as propriedades top, right, bottom e left.

5. position: sticky (O efeito chiclete)

É uma mistura genial entre o relative e o fixed. O elemento se comporta como relative (segue o fluxo da página) até que a rolagem chegue até ele. Nesse momento, ele "gruda" na tela como se fosse fixed. Quando a seção dele acaba, ele volta a rolar normalmente.

Onde é usado: Menus de navegação superiores que grudam no topo do site quando você rola a página para baixo.

É necessário você definir a altura da tela que ele deverá grudar! Exemplo: top: 0px. Essa classificação é obrigatória!

O código da imagem de como deixar vídeos responsivos (em conhecimentos/modulo2) mostra um truque clássico de web design para deixar vídeos responsivos (com a proporção perfeita de 16:9).

- A div.video ganha position: relative apenas para dizer ao navegador: "Eu sou o chefe e a referência de espaço aqui".
- O iframe ganha position: absolute com top: 5% e left: 5% para que ele flutue exatamente no centro da div controladora, ocupando 90% do espaço dela independentemente do tamanho da tela do celular ou computador do usuário!

]Inserção de ícones via CSS

Para inserir ícones via CSS, por exemplo pelo atributo "content", o link do ícone deve estar entre aspas simples e com uma barra invertida antes dele. É muito utilizado em *pseudo-element* como o "::after" e o "::before". Exemplo:

```
4  
5  ▾ a.externo::after {  
6    content: '\1F517'; /*colocar ícones via CSS*/  
7  }
```

Transition

O atributo *transition* deve ser colocado no elemento em que ele fará efeito ou no seu pseudo-class. Exemplo (ou vice-versa):

```
a{color: white; background-color: green; transition: color 1s;}
```

```
a:hover{color: green; background-color: white;}
```

Você também pode usar o **transition-duration**.

“Div p” e “Div > p”

Quando colocamos "main > p", os parágrafos que seram alterados são só aqueles que estiverem dentro da div "main", se tiver outra div dentro da div "main" que contenha parágrafos, eles não serão alterados. Porém, se você coloca "main p", todos os <p>, inclusive os que estiverem em sub-divs de "main", serão alterados.

Line-height

Altura das linhas. Isso ajuda para que quando o usuário termine de ler uma linha, seu olho não volte para o começo dela.

List-style-type

Esse atributo deve estar dentro de um elemento do CSS e ele é responsável por configurar um tipo para o marcador da sua lista.

Columns

Separa seu elemento em colunas (muito usado em listas).

00A0

O "00A0" é o caractere " " do HTML, que significa "espaço sem quebra". Com ele, você dá um espaço na linha sem quebrá-la. Mas esse caractere **NÃO** é compatível com todos os navegadores.

Se o navegador chegar ao fim da linha do monitor e encontrar um espaço comum entre duas palavras, ele vai quebrar a linha ali, jogando a segunda palavra para baixo. Se você usar o 00A0 entre elas, o navegador é forçado a mantê-las **juntas na mesma linha**, custe o que custar. É bom para coisas como 50km, 200m.

No HTML, se você apertar a barra de espaço 10 vezes seguidas no meio de um parágrafo, o navegador vai reduzir tudo para **apenas 1 espaço em branco**. Se você colocar o caractere 00A0 repetidas vezes, o navegador respeitará cada um deles e criará um grande espaçamento vazio.